

ΠΑΝΕΠΙΣΤΗΜΙΟ ΠΕΛΟΠΟΝΝΗΣΟΥ
ΠΟΛΥΤΕΧΝΙΚΗ ΣΧΟΛΗ
ΤΜΗΜΑ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ ΚΑΙ
ΜΗΧΑΝΙΚΩΝ ΥΠΟΛΟΓΙΣΤΩΝ
Επεξεργασία Ήχου και Μουσικής



Σχεδιασμός και Ανάπτυξη Διεπαφής
Φωνητικού Ελέγχου για Ρομποτικά
Συστήματα (VIRSI)

Γεώργιος Πεγιάζης 21081
Βασίλειος Τούμπας 21105

ΕΠΙΒΛΕΠΩΝ: Παναγιώτης Ζέρβας, Επίκουρος Καθηγητής

Σελίδα αφιέρωσης ή αποφθέγματος
(η σελίδα είναι προαιρετική)

Πίνακας Περιεχομένων

1. Εισαγωγή	3
1.1 Ερευνητικό Πλαίσιο	3
1.2 Παρουσίαση του προβλήματος	3
1.3 Επίλυση του προβλήματος	3
2. Σκοπός και Στόχοι.....	4
3. Γνώσεις που αξιοποιήθηκαν	5
3.1 Επιστημονικές γνώσεις.....	5
3.2 Τεχνικές γνώσεις.....	5
4. Η έννοια της φωνής	5
4.1 Το σήμα της φωνής.....	5
4.2 Ιδιότητες της φωνής και της κυματομορφής φωνής.....	5
4.3 Επεξεργασία σήματος φωνής	7
4.3.1 Συμπίεση σήματος.....	7
5. Μονάδα αναγνώρισης φωνής (VRM).....	10
5.1 Περιγραφή της VRM.....	10
5.2 Μονοκατευθυντικό μικρόφωνο MEMS	10
5.3 Τρόπος λειτουργίας της VRM.....	11
5.4 Προπόνηση της VRM V3.....	11
5.4.1 Εντολή <i>train</i>	12
5.4.2 Εντολή <i>load</i>	12
5.4.3 Εντολές <i>clear & record</i>	12
6. Μικροελεγκτές ESP32	13
6.1 Χαρακτηριστικά της ESP32	13
6.2 Πρωτόκολλο ασύρματης επικοινωνίας ESP-NOW	14
6.2.1 Χαρακτηριστικά του ESP-NOW.....	15
6.2.2 Μορφή Πλαισίου	15
6.2.3 Βασικές συναρτήσεις	17
6.2.4 Παράδειγμα επικοινωνίας μεταξύ δύο συσκευών.....	18
7. Υλοποίηση της VIRSI	21
7.1 Υλοποίηση της VIRSI	21
7.1.1 Υλικά	21
7.1.2 ESP-S3-WROOM-1N16R8	22
7.1.3 INMP441 I ² S μικρόφωνο	22
7.1.4 Ενισχυτής MAX98357.....	24
7.1.5 Κυκλώματα Sender & Receiver	25

8. Παρατηρήσεις.....	27
9. Πηγές.....	28

1. Εισαγωγή

1.1 Ερευνητικό Πλαίσιο

Η ανάπτυξη συστημάτων διεπαφής ανθρώπου-μηχανής (HMI) αποτελεί έναν ραγδαία αναπτυσσόμενο διεπιστημονικό τομέα. Στη ρομποτική, ο φωνητικός έλεγχος αποκτά αυξανόμενη σημασία, διευκολύνοντας την αλληλεπίδραση και ενισχύοντας τη λειτουργικότητα. Η τεχνολογία φωνητικής αναγνώρισης επιτρέπει τη δημιουργία ευφυών συστημάτων που ανταποκρίνονται σε λεκτικές εντολές, με εφαρμογές στη βιομηχανία, την οικιακή ρομποτική και τις βοηθητικές τεχνολογίες.

1.2 Παρουσίαση του προβλήματος

Η παρούσα εργασία επικεντρώνεται στον σχεδιασμό και την ανάπτυξη ενός συστήματος φωνητικού τηλεχειρισμού για ρομποτικά συστήματα (**VIRSI – Voice Interface for Robotic Systems Interaction**), αξιοποιώντας το **Voice Recognition Module V3 , ESP-32** μικροελεγκτές και κάποιους απαραίτητους αισθητήρες. Στόχος είναι η δημιουργία μιας απλής αλλά και αξιόπιστης φωνητικής διεπαφής που μεταφράζει λεκτικές εντολές σε σήματα ελέγχου. Η υλοποίηση περιλαμβάνει τη σύνδεση του αναγνωριστικού φωνής με έναν ESP μικροελεγκτή, ο οποίος επεξεργάζεται τις φωνητικές εντολές και επικοινωνεί με δεύτερο μικροελεγκτή για την εκτέλεση των λειτουργιών μέσω του πρωτοκόλλου ασύρματης επικοινωνίας **ESP-NOW**. Η ανάγκη για φυσικό έλεγχο είναι σημαντική σε περιβάλλοντα όπου οι παραδοσιακές μέθοδοι είναι περιορισμένες.

1.3 Επίλυση του προβλήματος

Η προτεινόμενη λύση αξιοποιεί οικονομικά προσιτές τεχνολογίες για τη δημιουργία ενός ευέλικτου συστήματος φωνητικού ελέγχου. Η χρήση του Voice Recognition Module V3 και των ESP μικροελεγκτών παρέχει τη δυνατότητα ερμηνείας φωνητικών εντολών και μετάδοσής τους στα ρομποτικά υποσυστήματα. Το σύστημα είναι επεκτάσιμο και επαναχρησιμοποιήσιμο, προσαρμοζόμενο σε διάφορα ρομποτικά περιβάλλοντα, και απευθύνεται σε σενάρια όπου απαιτείται hands-free χειρισμός ή αυξημένη προσβασιμότητα, αποτελώντας παράλληλα μια εκπαιδευτική πλατφόρμα.

2. Σκοπός και Στόχοι

Ο βασικός σκοπός της παρούσας εργασίας είναι η εφαρμογή και αξιοποίηση τεχνικών επεξεργασίας ήχου στο πλαίσιο της φωνητικής αλληλεπίδρασης με ρομποτικά συστήματα. Μέσω της ενσωμάτωσης τεχνολογιών αναγνώρισης φωνής, επιχειρείται η ανάπτυξη ενός συστήματος που μπορεί να λαμβάνει, να επεξεργάζεται και να αποκωδικοποιεί ηχητικά σήματα σε πραγματικό χρόνο, επιτρέποντας τον φυσικό έλεγχο ρομποτικών υποσυστημάτων. Η εργασία συνδυάζει βασικές αρχές της επεξεργασίας φωνής με πρακτική εφαρμογή σε ενσωματωμένα συστήματα και επικοινωνιακές πλατφόρμες.

Οι επιμέρους στόχοι της εργασίας είναι οι εξής:

- Μελέτη και κατανόηση βασικών εννοιών επεξεργασίας ήχου και φωνής, καθώς και των παραμέτρων που επηρεάζουν την ποιότητα της φωνητικής αναγνώρισης.
- Σχεδίαση και υλοποίηση συστήματος φωνητικού ελέγχου βασισμένου στο Voice Recognition Module V3, με έμφαση στην αναγνώριση και αποκωδικοποίηση φωνητικών εντολών.
- Ανάπτυξη συστήματος συλλογής και επεξεργασίας ακουστικών σημάτων σε επίπεδο ενσωματωμένων συσκευών.
- Διασύνδεση του φωνητικού υποσυστήματος με ESP μικροελεγκτές για την ασύρματη μετάδοση εντολών μέσω του πρωτοκόλλου ασύρματης επικοινωνίας ESP-NOW.
- Δημιουργία αρθρωτής και επεκτάσιμης αρχιτεκτονικής για την υποστήριξη διαφορετικών ρομποτικών εφαρμογών.
- Πειραματική αξιολόγηση της απόδοσης του συστήματος επεξεργασίας ήχου σε συνθήκες θορύβου και διαφορετικών φωνών.
- Ανάδειξη της εκπαιδευτικής διάστασης του έργου, στο πλαίσιο του μαθήματος «Επεξεργασία Ήχου», μέσω μιας πρακτικής και διαδραστικής εφαρμογής.

3. Γνώσεις που αξιοποιήθηκαν

3.1 Επιστημονικές γνώσεις

Για την υλοποίηση της διεπαφής φωνητικών εντολών για ρομποτικά συστήματα αξιοποιήσαμε τις θεωρητικές μας γνώσεις τόσο στα αναλογικά όσο και στα ψηφιακά ηλεκτρονικά κυκλώματα, στα ασύρματα δίκτυα επικοινωνίας (π.χ. Wi-Fi), στην ανάπτυξη αλγορίθμων, στην επεξεργασία αναλογικών και ψηφιακών σημάτων με χρήση του MATLAB, στα μαθηματικά, τη φυσική, στον διαδικασιακό και στον αντικειμενοστραφή προγραμματισμό και τέλος και στα δίκτυα υπολογιστών.

3.2 Τεχνικές γνώσεις

Για την υλοποίηση της διεπαφής φωνητικών εντολών για ρομποτικά συστήματα υλοποιήσαμε τον προγραμματισμό μέσω του Arduino IDE σε γλώσσα C/C++ ,τόσο για την ασύρματη επικοινωνία όσο και για την κίνηση του ρομπότ, ενώ με χρήση του MATLAB, μπορέσαμε να επεξεργαστούμε με διάφορους τρόπους τα σήματα φωνής. Επίσης, αξιοποιήσαμε τις τεχνικές μας γνώσεις στο σχεδιασμό ηλεκτρονικών κυκλωμάτων.

4. Η έννοια της φωνής

4.1 Το σήμα της φωνής

Σύμφωνα με την **θεωρία πληροφορίας του Shannon**, ένα μήνυμα που αναπαρίσταται ως μία ακολουθία διακριτών συμβόλων, μπορεί να ποσοτικοποιηθεί με βάση το **πληροφοριακό του περιεχόμενο** σε bit, όπου ο ρυθμός μετάδοσης της πληροφορίας μετριέται σε **bits/δευτερόλεπτα (bps)**. Κατά την ομιλία, η πληροφορία που μεταδίδεται **κωδικοποιείται** σε μορφή **συνεχώς μεταβαλλόμενης (αναλογικής) κυματομορφής**, η οποία μπορεί να μεταδοθεί, εγγραφεί (αποθηκευτεί), επεξεργαστεί και τελικώς να αποκωδικοποιηθεί από έναν ακροατή. Η στοιχειώδης αναλογική μορφή του μηνύματος είναι μια ακουστική κυματομορφή την οποία καλούμε **σήμα φωνής**.

4.2 Ιδιότητες της φωνής και της κυματομορφής φωνής

Το σήμα φωνής έχει τις παρακάτω εγγενείς ιδιότητες:

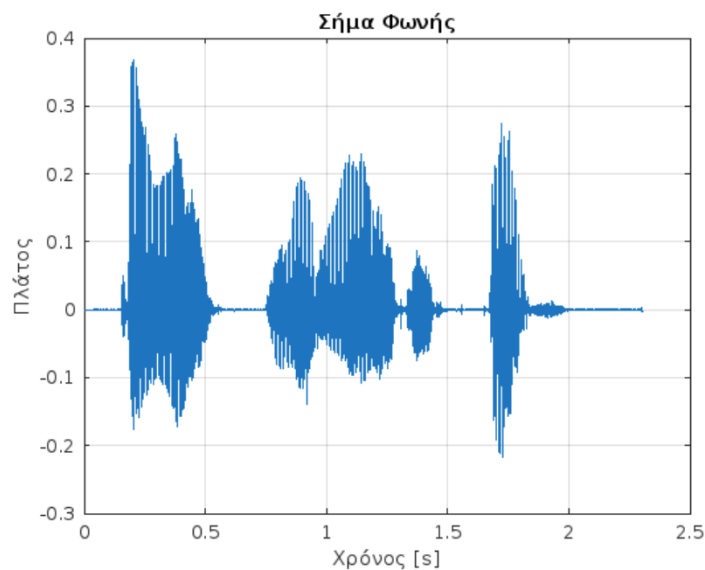
- Πρόκειται για μια αλληλουχία διαρκώς μεταβαλλόμενων ήχων.
- Οι ιδιότητες της κυματομορφής του σήματος εξαρτώνται από τους ήχους που παράγονται και κωδικοποιούν το περιεχόμενο του μηνύματος.
- Οι ιδιότητες του σήματος εξαρτώνται από το πλαίσιο στο οποίο παράγονται οι ήχοι, από τους ήχους δηλαδή που παράγονται πριν και μετά τον τρέχοντα ήχο. Το φαινόμενο αυτό είναι γνωστό ως **συνάρθρωση (co-articulation)** και είναι αποτέλεσμα του μηχανισμού ελέγχου φωνής που προετοιμάζεται για τους επόμενους ήχους, καθώς παράγει τον τρέχον ήχο, μεταβάλλοντας ανάλογα και τις ιδιότητές του.

- Η κατάσταση των φωνητικών χορδών και η θέση, το σχήμα και το μέγεθος των διάφορων αρθρωτών (π.χ. γλώσσα, χείλη, δόντια κτλ.) αλλάζουν προοδευτικά με τον χρόνο, παράγοντας έτσι τους επιθυμητούς ήχους.

Παρακάτω, ακολουθεί ένας ενδεικτικός κώδικας με τον οποίο μπορούμε να απεικονίσουμε και να δούμε πως μοιάζει ένα σήμα φωνής. Στην *Εικόνα 1* απεικονίζεται το ίδιο το σήμα φωνής.

```
% Παράδειγμα ενός σήματος φωνής
[y,Fs] = audioread("Command.wav");
%y: διάνυσμα των δειγμάτων
%Fs: συχνότητα δειγματοληψίας (Hz)
t = (0:length(y)-1)/Fs; % χρονικός άξονας (sec)

% Απεικόνιση του σήματος στο πεδίο του χρόνου
('Name','Time Domain Signal');
plot(t, y);
xlabel('Χρόνος [s]');
ylabel('Πλάτος');
title('Σήμα Φωνής');
grid on;
```



Εικόνα 1: Σήμα εντολής «Turn the lights off».

Το σχήμα στην *Εικόνα 1* απεικονίζει τη μεταβολή του πλάτους της φωνητικής κυματομορφής σε συνάρτηση με το χρόνο, για την φράση «**Turn the lights off**», η οποία ηχογραφήθηκε μέσω αντίστοιχης εφαρμογής εγγραφής φωνής στο κινητό, και έπειτα αποθηκεύτηκε ως **.wav** αρχείο. Η συνάρτηση **audioread** μας επιτρέπει να «διαβάσουμε» τα δεδομένα από οποιοδήποτε **.wav** αρχείο και μας επιστρέφει τα δείγματα δεδομένων (παράμετρος **y**) και το ρυθμό με τον οποίο αυτά τα δεδομένα λήφθηκαν (μεταβλητή **Fs**). Μπορούμε να δούμε ότι στο συγκεκριμένο σήμα εμφανίζονται μικρές παύσεις, κάτι το οποίο είναι απολύτως λογικό να συμβαίνει, μιας και έχουμε να κάνουμε με την ανθρώπινη ομιλία. Ταυτόχρονα, είμαστε σε θέση να δούμε και ότι στα σημεία στα οποία εκφωνούνται οι λέξεις, υπάρχουν έντονες διακυμάνσεις.

4.3 Επεξεργασία σήματος φωνής

Αν θέλουμε το σήμα φωνής μας να είναι όσο πιο γίνεται «τέλειο», μπορούμε γενικότερα να το επεξεργαστούμε με διάφορους τρόπους, αναλόγως το τι θέλουμε εμείς κάθε φορά.

4.3.1 Συμπίεση σήματος

Μια τεχνική που μπορούμε να εφαρμόσουμε σε ένα σήμα ήχου είναι αυτή της **συμπίεσης (compressor)**. Με την συμπίεση μπορούμε να ρυθμίσουμε την ένταση του σήματός μας να είναι όσο το δυνατόν πιο **σταθερή ακουστικά**, χωρίς να χάνουμε λέξεις, απλά **μειώνοντας τις μεγάλες διακυμάνσεις έντασης και ενισχύοντας τα πιο αδύναμα σημεία**. Ένας συμπίεστής χαρακτηρίζεται από τα εξής:

- **Κατώφλι (threshold)**: το όριο της έντασης πάνω από το οποίο ξεκινάει η συμπίεση. Μετρείται σε **dB**.
- **Λόγος συμπίεσης (ratio)**: το ποσοστό μείωσης της έντασης πέρα από το κατώφλι.
- **Χρόνος επίθεσης/απελευθέρωσης (attack/release time)**: χρόνοι που μας δείχνουν το πόσο γρήγορα ξεκινάει ή σταματάει η συμπίεση.
- **Τελική ενίσχυση (make-up gain)**: η τιμή της ενίσχυσης που προκύπτει μετά την συμπίεση, ώστε να φτάσει στην επιθυμητή ένταση.

Πώς λειτουργεί όμως ένας compressor;

Ο συμπίεστής υπολογίζει την **RMS** τιμή της έντασης για κάθε μικρό χρονικό πλαίσιο και ελέγχει αν το σήμα βρίσκεται πάνω από την τιμή του κατωφλίου. Η RMS τιμή για κάθε πλαίσιο $x[n]$, μεγέθους N , δίνεται από την σχέση:

$$RMS = \sqrt{\frac{1}{N} \sum_{n=1}^N x[n]^2}$$

Αν αυτό ισχύει, τότε μειώνει την ένταση με βάση το λόγο συμπίεσης. Για παράδειγμα, αν ο λόγος συμπίεσης είναι 4:1 (δηλ. ratio = 4dB), τότε για κάθε 4dB που βρίσκονται πάνω από το όριο, αυτά μειώνονται σε 1dB. Ύστερα, χρησιμοποιεί τους χρόνους επίθεσης και απελευθέρωσης, έτσι ώστε να μεταβαίνει ομαλά (**γρήγορη απόκριση-attack και πιο αργή επιστροφή-release**). Για την υλοποίηση αυτή της ομαλής μετάβασης, ο συμπίεστής χρησιμοποιεί ένα **φίλτρο** το οποίο ονομάζεται **EMA filter (Exponential Moving Average)**. Το φίλτρο αυτό (στα ελληνικά ονομάζεται **φίλτρο εκθετικού κινούμενου μέσου όρου**), πρόκειται για ένα **μονοπολικό (first order) χαμηλοπερατό (low-pass) άπειρης απόκρισης παλμού (Infinite Impulse Response – IIR)** φίλτρο, το οποίο χρησιμοποιείται στην επεξεργασία σημάτων και δίνεται από την πολύ απλή μαθηματική σχέση:

$$y[n] = \alpha x[n] + (1 - \alpha)y[n - 1]$$

Όπου:

- $x[n]$ η είσοδος (το αρχικό σήμα),
- $y[n]$ η έξοδος (το εξομαλυσμένο σήμα) και
- **α** , ο **συντελεστής εξομάλυνσης (smoothing factor)** για τον οποίο ισχύει $\alpha \in (0, 1)$.

Αν ο α είναι **κοντά στο 1**, τότε το φίλτρο «ακολουθεί» γρήγορα τις αλλαγές στο σήμα εισόδου $x[n]$ (γρήγορο attack). Αν ο α τώρα είναι **κοντά στο 0**, τότε το φίλτρο διατηρεί («συγκρατεί») τον

προηγούμενο όρο της εξόδου ($y[n-1]$) και αλλάζει πολύ αργά (αργό release). Στην πραγματικότητα, ο συμπίεστής χρησιμοποιεί δύο τέτοια φίλτρα. Ένα με **μεγάλο α** και ένα με **μικρό α**.

Το τελευταίο στάδιο στην διαδικασία της συμπίεσης, είναι αυτό της **τελικής ενίσχυσης (make-up gain)**. Με την ολοκλήρωση της συμπίεσης, προστίθεται μια **σταθερή ενίσχυση σε dB**, έτσι ώστε σε περίπτωση που το σήμα μας είναι εν τέλει πιο χαμηλό, να αντισταθμίσουμε σε έναν βαθμό την απώλεια αυτή. Η σχέση που μας δίνει την τελική ενίσχυση είναι:

$$gain = 10^{\frac{G}{20}}$$

Όπου G η **make-up gain**.

Έτσι, το σήμα μας $x[n]$ με την ολοκλήρωση της συμπίεσης θα δίνεται από τη σχέση:

$$x[n]' = x_{compressed}[n] \cdot gain$$

Τι κερδίζουμε από την διαδικασία της συμπίεσης:

- Μέσω της συμπίεσης, η ομιλία (π.χ. σε ένα βίντεο) γίνεται **πιο συνεκτική και κατανοητή**.
- Οι χαμηλόφωνες λέξεις δεν χάνονται από το σήμα και γίνονται πιο εύκολα αντιληπτές από το ανθρώπινο αυτί.
- Τέλος, οι έντονες λέξεις δεν παραμορφώνουν την ποιότητα του σήματός μας.

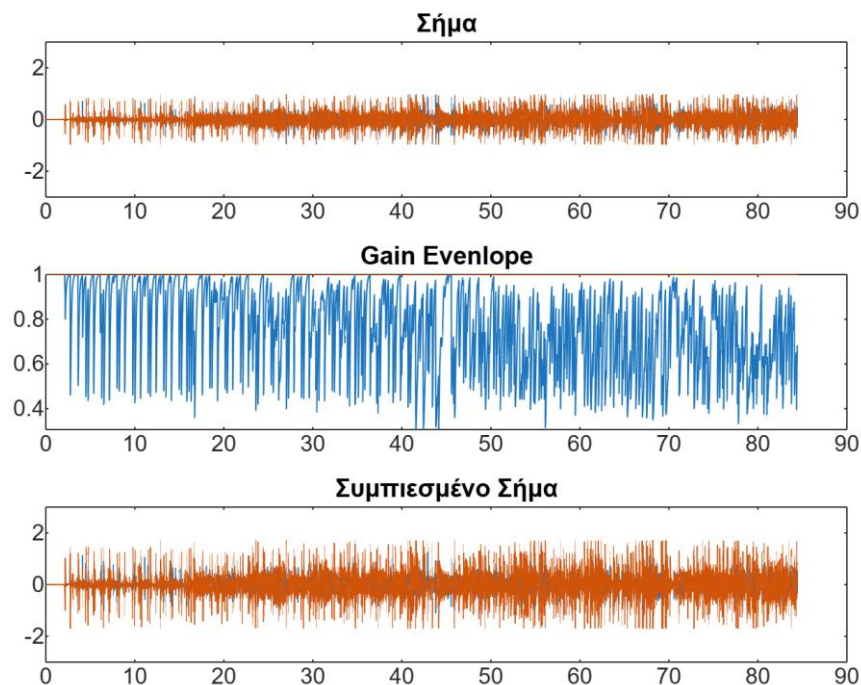
Παρακάτω, ακολουθεί ένα παράδειγμα υλοποίηση ενός compressor με χρήση του MATLAB και ενός αρχείου μουσικής (music.wav).

```
% Παράδειγμα σήματος φωνής
[signal,Fs] = audioread("music.WAV");
% Μεταβλητές του συμπίεστή
threshold_dB = -20;
ratio = 4;
attackTime = 0.01;
releaseTime = 0.1;
makeUpGain_dB = 5;
% Μετατροπή του κατωφλίου στην γραμμική κλίμακα
thres = 10^(threshold_dB/20);
makeUpGain = 10^(makeUpGain_dB/20);
% Υπολογισμός RMS για κάθε πλαίσιο
frame_N = round(Fs*0.01); % Κάθε πλαίσιο έχει μέγεθος 10ms
rms = sqrt(movmean(signal.^2 , frame_N));
% Εφαρμογή της συμπίεσης
gain = ones(size(rms));
for i = 1:length(rms)
    if rms(i) > thres
        gain(i) = (rms(i)/thres)^((1-ratio)/ratio);
    end
end
% Ορισμοί των smoothing factors
a1 = exp(-1/(Fs*attackTime));
a2 = exp(-1/(Fs*releaseTime));
new_gain = gain;
for i = 2:length(gain)
    if gain(i) < new_gain(i-1)
        new_gain(i) = a1 * new_gain(i-1) + (1-a1)*gain(i);
    else
```

```

        new_gain(i) = a2 * new_gain(i-1) + (1-a2)*gain(i);
    end
end
% Τελική ενίσχυση
new_signal = signal .* new_gain * makeUpGain;
% Απεικόνιση γραφημάτων
t = (0:length(signal)-1) / Fs;
figure;
subplot(3,1,1); plot(t,signal);title('Σήμα');ylim([-3 3]);
subplot(3,1,2); plot(t, new_gain); title('Gain Evenlope');
subplot(3,1,3);plot(t,new_signal);title('Συμπίεσμένο Σήμα'); ylim([-3 3]);
% Αναπαραγωγή του συμπίεσμένου σήματος
soundsc(new_signal,Fs);

```



Εικόνα 2: Συμπίεση σήματος μουσικής.

Όπως φαίνεται και από το πρώτο γράφημα στην Εικόνα 2, στο σήμα μας εμφανίζονται αρκετές μεγάλες διακυμάνσεις πλάτους, κάτι που είναι λογικό μιας και μιλάμε για μουσική. Στο δεύτερο γράφημα, φαίνεται η μεταβολή του gain που υπολόγισε ο συμπίεστής στην πάροδο του χρόνου, προτού εφαρμοστεί στο σήμα η τελική ενίσχυση μέσω του make-up gain. Βλέπουμε ότι όταν το σήμα ξεπερνάει την τιμή του κατωφλίου, η καμπύλη ενίσχυσης πέφτει, πράγμα που σημαίνει ότι ο συμπίεστής μειώνει τα κομμάτια του σήματος στα οποία η μουσική ακούγεται πιο έντονα. Στην άλλη τώρα περίπτωση, δηλαδή όταν το σήμα δεν ξεπερνάει την τιμή κατωφλίου, η καμπύλη της ενίσχυσης βρίσκεται **κοντά στο 1**. Αυτό σημαίνει ότι ο compressor **δεν επηρεάζει τα σημεία στα οποία η ένταση είναι χαμηλή**. Τέλος, στο τρίτο γράφημα που απεικονίζεται το συμπίεσμένο σήμα, οι χαμηλές περιοχές έχουν ενισχυθεί λόγω του make-up gain, με αποτέλεσμα οι μικρές διακυμάνσεις που πριν ήταν σχεδόν «παύση» πλέον να γίνονται ηχητικά αισθητές. Συνολικά, ωστόσο, το σήμα ακούγεται πιο συνεκτικό χωρίς απότομα ανεβάσματα της έντασης.

5. Μονάδα αναγνώρισης φωνής (VRM)

5.1 Περιγραφή της VRM

Η VRM (Voice Recognition Module V3) της Elechouse είναι μια **αυτόνομη μονάδα φωνητικής αναγνώρισης**, σχεδιασμένη ώστε να μπορεί να ενσωματωθεί πολύ εύκολα σε συστήματα τα οποία δρουν μέσω φωνητικών εντολών από τον χρήστη, είτε με την βοήθεια μικροελεγκτών όπως Arduino, ESP ή STEM32, είτε μικροεπεξεργαστών όπως Raspberry Pi, χωρίς να απαιτείται κάποιου είδους σύνδεσης με εξωτερικό υπολογιστή ή ακόμη και σύνδεση στο cloud.



Εικόνα 3: Voice Recognition Module V3 by Elechouse.

Η μονάδα φωνητικής αναγνώρισης λειτουργεί στα **3.3V-5V στο DC (συνεχές ρεύμα)**, είναι μικρή σε μέγεθος (31mm x 50mm), πράγμα που την καθιστά ιδανική για χρήση σε ρομποτικά συστήματα για τον έλεγχο κίνησης, για εντολές τύπου “one-shot” σε συστήματα IoT, για χρήσεις home automation (έξυπνο σπίτι) κ.ά.

Τέλος, η πλακέτα διαθέτει δύο τρόπους ελέγχου:

- μέσω **σειριακής θύρας (serial port)** και
- με χρήση **γενικών ακίδων εισόδου (general input pins – GIP)**.

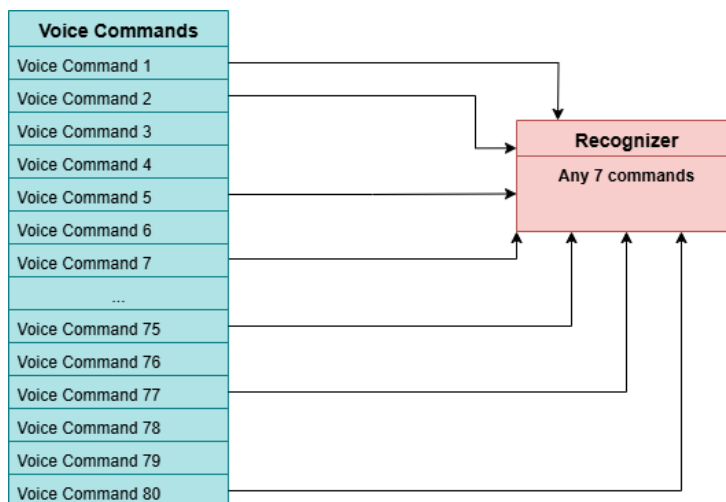
5.2 Μονοκατευθυντικό μικρόφωνο MEMS

Η πλακέτα είναι εξοπλισμένη με ενσωματωμένο **μονοκατευθυντικό μικρόφωνο τύπου MEMS (Micro-Electro-Mechanical Systems)**. Το μικρόφωνο είναι μικρό σε μέγεθος, αλλά ταυτόχρονα χαμηλό σε κατανάλωση. Είναι σχεδιασμένο να δέχεται τον ήχο κυρίως από την μία κατεύθυνση (**uni-directional**), μειώνοντας τον περιβαλλοντικό θόρυβο. Ένα τέτοιο μικρόφωνο ενδείκνυται για φωνητικές εντολές μικρών αποστάσεων ($\approx 20\text{-}50$ εκατοστά για την βέλτιστη ακρίβεια – περίπου 99% σε ιδανικές συνθήκες περιβάλλοντος). Ωστόσο, αν χρειαζόμαστε **καλύτερη ποιότητα ή μεγαλύτερη απόσταση φωνής**, τότε μπορούμε να συνδέσουμε ένα εξωτερικό μικρόφωνο χρησιμοποιώντας το **MIC IN pin**, που βρίσκεται δίπλα στις εισόδους για την τάση τροφοδοσίας

(VCC) και την γείωση (GND). Επιπλέον, στην VRM μέσω της υποδοχής audio jack, μπορούμε να συνδέσουμε και ένα εξωτερικό μικρόφωνο, γνωστό και ως **gooseneck**, το οποίο είναι εξοπλισμένο με σφουγγαράκι για καλύτερη απόσβεση του θορύβου που δέχεται το μικρόφωνο (π.χ. από την αναπνοή ή τον αέρα).

5.3 Τρόπος λειτουργίας της VRM

Η πλακέτα αναγνώρισης φωνής υποστηρίζει **εντολές εκπαίδευσης (training)** από τον χρήστη. Μπορεί να εκπαιδευτεί τοπικά με μέχρι **80 λέξεις/εντολές**, έχοντας την δυνατότητα να υποστηρίξει μέχρι **7 λέξεις/εντολές ενεργές τη φορά**, δηλαδή η VRM «ακούει» και αναγνωρίζει μόνο αυτές τις επτά. Οι εντολές που μπορεί να δεχθεί και να αποθηκεύσει σε κάθε θέση μνήμης πρέπει να είναι **είτε δύο λέξεις με μέγιστη διάρκεια 1.5 δευτερόλεπτα (1500ms) είτε μία μόνο λέξη**. Η κάθε εντολή, αποθηκεύεται στο εσωτερικό flash της πλακέτας με ένα συγκεκριμένο id. Κάθε φορά που αναγνωρίζει με επιτυχία μια από τις ενεργές λέξεις/εντολές, επιστρέφει τον αντίστοιχο id αριθμό της λέξης/εντολής αυτής, μέσω UART επικοινωνίας (μέσω δηλαδή των Tx/Rx). Η επιλογή της γλώσσας με την οποία θα «προπονηθεί» η VRM δεν παίζει κάποιο ρόλο, αφού η λειτουργία της πλακέτας βασίζεται **εξ' ολοκλήρου στον ήχο** και όχι σε κάποια συγκεκριμένη γλώσσα.



Εικόνα 4: Διάγραμμα απεικόνισης λειτουργίας της VRM V3

Όλες οι φωνητικές εντολές αποθηκεύονται σε ένα **κοινό για όλες group**. Όπως φαίνεται και στο διάγραμμα στην *Εικόνα 4*, στην πλακέτα βρίσκεται ο **αναγνωριστής (recognizer)**, ο οποίος δρα ως ένα κοντέινερ στο οποίο φορτώνονται οι φωνητικές εντολές εκτέλεσης που μπορούν να αναγνωριστούν ταυτόχρονα. Στην ουσία λειτουργεί σαν μια λίστα στην οποία αποθηκεύονται τα στοιχεία που θέλουμε κάθε φορά.

5.4 Προπόνηση της VRM V3

Για την προπόνηση της VRM χρειάζονται να γίνουν τα ακόλουθα βήματα:

- Συνδέουμε κατάλληλα την VRM V3 με τον μικροελεγκτή μας
- Λήψη βιβλιοθήκης **VoiceRecognitionV3**.
- Αφού ανοίξουμε το Arduino IDE -> vr_sample_train (File -> Examples -> > VoiceRecognitionV3 -> vr_sample_train).
- Πατάμε το Upload.
- Αφού ανέβει στην πλακέτα μας, ανοίγουμε το Serial Monitor και ορίζουμε το baud rate στα 115200bps.

Με την ολοκλήρωση των παραπάνω βημάτων, μας εμφανίζεται μέσω του COM port που έχουμε συνδέσει τον μικροελεγκτή μας ένα παράθυρο μέσα από το οποίο μπορούμε να κάνουμε την προπόνηση. Η διαδικασία ξεκινάει αρχικά πληκτρολογώντας μια από τις εντολές που φαίνονται παρακάτω.

COMMAND	FORMAT	EXAMPLE	Comment
train	train (r0) (r1)...	train 0 2 45	Train records
load	load (r0) (r1) ...	load 0 51 2 3	Load records
clear	clear	clear	remove all records in Recognizer
record	record / record (r0) (r1)...	record / record 0 79	Check record train status
vr	vr	vr	Check recognizer status
getsig	getsig (r)	getsig 0	Get signature of record (r)
sigtrain	sigtrain (r) (sig)	sigtrain 0 ZERO	Train one record(r) with signature(sig)
settings	settings	settings	Check current system settings

Εικόνα 5: Εντολές για την προπόνηση της VRM V3

5.4.1 Εντολή train

Με την εντολή **train (0)** ξεκινάμε την διαδικασία προπόνησης της εντολής με id 0 κ.ο.κ. Όταν πατήσουμε Enter θα μας εμφανιστεί ένα μήνυμα στο παράθυρο που θα μας λέει «**Speak now**» και το ledάκι της VRM V3 θα αλλάζει χρώμα. Αυτό σημαίνει ότι εκείνη την στιγμή θα πρέπει να πούμε την εντολή που θέλουμε. Αφού την ξαναπούμε άλλη μια φορά, τότε θα μας εμφανιστεί ένα μήνυμα το οποίο είτε θα μας δείχνει ότι η διαδικασία απέτυχε και θα πρέπει να την επαναλάβουμε είτε ότι στέφθηκε με επιτυχία.

5.4.2 Εντολή load

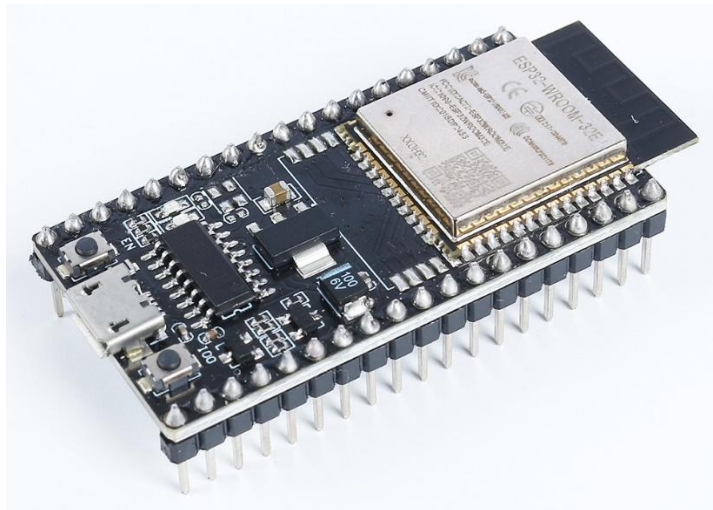
Με την εντολή **load (0)** φορτώνουμε από την flash της VRM V3 την εντολή με id 0 έτσι ώστε να είναι έτοιμη για αναγνώριση. Ομοίως και για τις υπόλοιπες εντολές. Όταν πατήσουμε Enter τότε η VRM V3 θα περιμένει από εμάς να ακούσει την εντολή που έχουμε προπονήσει στο record με id 0 και αφού την ακούσει και την αναγνωρίσει θα μας επιστρέψει ένα αντίστοιχο μήνυμα επαλήθευσης στην οθόνη μας.

5.4.3 Εντολές clear & record

Με την εντολή **record** μπορούμε να δούμε όλες τις προπονημένες - και μη προπονημένες - εγγραφές στην flash της VRM V3. Για παράδειγμα με την εντολή **record 0** μπορούμε να δούμε την κατάσταση της εγγραφής με id 0. Με την εντολή **clear** μπορούμε να διαγράψουμε όλες τις εντολές από τον recognizer και να ξεκινήσουμε νέα προπόνηση από την αρχή.

6. Μικροελεγκτές ESP32

6.1 Χαρακτηριστικά της ESP32



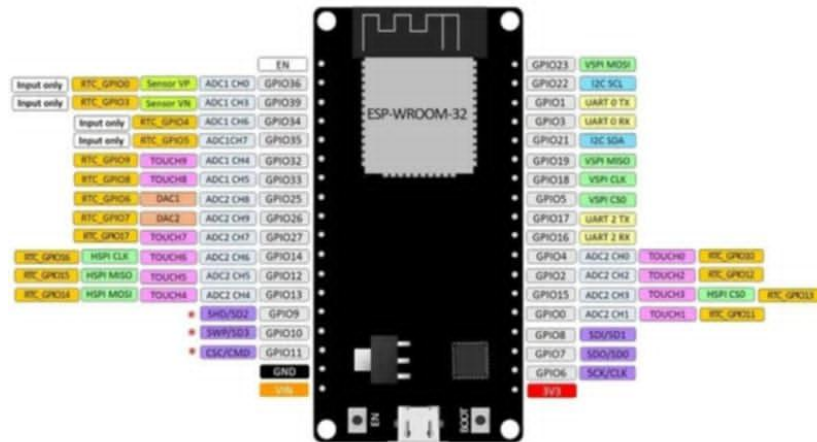
Εικόνα 6: ESP32-WROOM-32E

Το **ESP32** είναι μία οικογένεια **μικροελεγκτών (microcontrollers)**, χαμηλού κόστους και ενεργειακής αποδοτικότητας, με δυνατότητες σύνδεσης στα ασύρματα δίκτυα **Wi-Fi και Bluetooth**. Ένας μικροελεγκτής ESP32, έχει ενσωματωμένα απαραίτητα εξαρτήματα έτσι ώστε να μπορεί να επιτευχθεί η ασύρματη επικοινωνία και μετάδοση δεδομένων, όπως για παράδειγμα διακόπτες κεραίας, ενισχυτές ισχύος, φίλτρα, δέκτες χαμηλού θορύβου κ.ά. Συνήθως, διατίθεται ως ένα εξάρτημα σε αναπτυξιακά κιτ τα οποία περιλαμβάνουν διάφορες ακίδες γενικού σκοπού εισόδου και εξόδου (τα λεγόμενα GPIO). Σχεδιάστηκε από την TSMC και αποτελεί τον διάδοχο του μικροελεγκτή ESP8266.

Μερικά από τα κυριότερα χαρακτηριστικά ενός ESP32 μικροελεγκτή είναι τα ακόλουθα:

- **Μνήμη:** 520 KiB RAM, 448 KiB ROM.
- **Ασύρματη σύνδεση σε:** 1) **Wi-Fi: 802.11 b/g/n** και
2) **Bluetooth: v4.2 BR/EDR & BLE.**
- **Περιφερειακές διεπαφές:**
 - 34 προγραμματιζόμενα **GPIOs**
 - 2x 8-bit **DAC**
 - **PWM LED** (έως 16 κανάλια)
 - **PWM** για κινητήρα
 - **Υπέρυθρο τηλεχειρισμός** (Tx/Rx έως 8 κανάλια)
 - **Μετρητή παλμών** κ.ά.

Τέλος, παρέχει πλήρη υποστήριξη των προτύπων ασφαλείας **IEEE 802.11**, συμπεριλαμβανομένων των **WPA, WPA2, WP3** (με βάση την έκδοση) και του **WAPI** (Υποδομή Αυθεντικοποίησης και Ιδιωτικότητας WLAN) κ.ά.



Εικόνα 7:ESP32-WROOM-32 Pinout

Οι μικροελεγκτές ESP32 χρησιμοποιούνται σε πολλούς τομείς λόγω της δυνατότητας ασύρματης επικοινωνίας, της χαμηλής τους κατανάλωσης και την εύκολη επεξεργασία δεδομένων. Μερικοί από αυτούς τομείς είναι:

- **Διαδίκτυο των Πραγμάτων (Internet of Things – IoT):** χρησιμοποιούνται για παράδειγμα στα έξυπνα σπίτια μαζί με αισθητήρες και αυτοματισμούς.
- **Βιομηχανικός αυτοματισμός:** χρησιμοποιούνται για την παρακολούθηση και τον έλεγχο των μηχανών ή διάφορων άλλων διαδικασιών.
- **Ρομποτική:** λόγω των PWM καναλιών, δίνεται η δυνατότητα ελέγχου των κινητήρων σε ένα ρομπότ και χρησιμεύουν τόσο στον προγραμματισμό των αισθητήρων όσο και στην μεταξύ τους επικοινωνία.
- **Φορητές συσκευές και wearables:** Συναντάται σε ρολόγια, μετρητές υγείας, συσκευές παρακολούθησης κ.ά.
- **DIY projects:** Χρησιμοποιούνται σε εφαρμογές ανοικτού κώδικα και εκπαιδευτικά έργα.

Γενικότερα, η ευελιξία τους είναι αυτό που τους καθιστά ιδανικούς τόσο για μεγάλα επαγγελματικά έργα όσο και για προσωπικές μας μικρότερες εφαρμογές.

6.2 Πρωτόκολλο ασύρματης επικοινωνίας ESP-NOW

Το **ESP-NOW** είναι ένα πρωτόκολλο ασύρματης επικοινωνίας που βασίζεται στο επίπεδο σύνδεσης δεδομένων το οποίο μειώνει τα πέντε από τα επτά επίπεδα του μοντέλου **OSI (Open Systems Interconnection)** σε ένα μόνο επίπεδο. Με αυτό τον τρόπο, τα δεδομένα δεν χρειάζεται να μεταδίδονται μέσω του επιπέδου δικτύου (Network Layer), του επιπέδου μεταφοράς (Transport

Layer), του επιπέδου συνεδρίας (Session Layer), του επιπέδου παρουσίασης (Presentation Layer) και του πεδίου εφαρμογής (Application Layer). Τα δεδομένα, το λοιπόν, μπορούν να μεταδοθούν **μόνο** μέσω του **φυσικού επιπέδου (Physical Layer)** και του **επιπέδου σύνδεσης δεδομένων (Data Link Layer)**. Έτσι, δεν υπάρχει και η ανάγκη για κεφαλίδες πακέτων ή προγράμματα αποσυμπίεσης σε κάθε επίπεδο, γεγονός που οδηγεί σε **γρήγορη απόκριση**, μειώνοντας την καθυστέρηση που προκαλείται από την απώλεια πακέτων σε δίκτυα με συμφόρηση. Στην *Εικόνα 7*, απεικονίζονται οι διαφορές μεταξύ των μοντέλων OSI και ESP-NOW.



Εικόνα 8: OSI & ESP-NOW Models

6.2.1 Χαρακτηριστικά του ESP-NOW

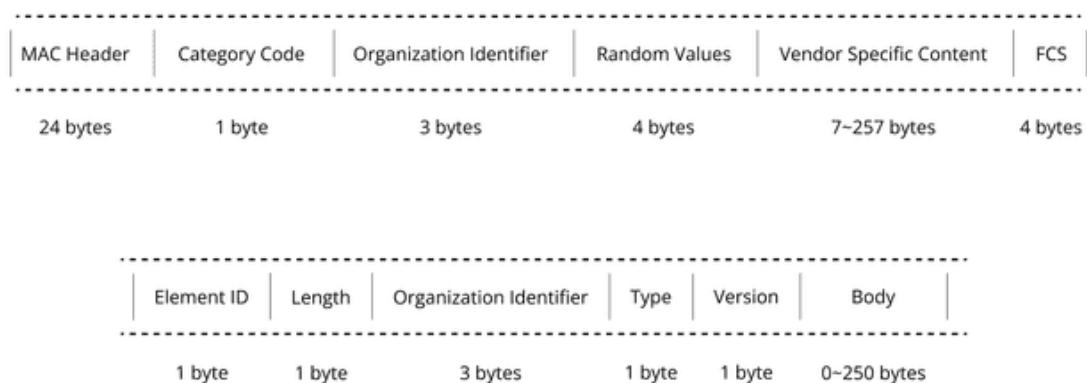
Το ESP-NOW ξεχωρίζει διότι:

- Μπορεί και συνυπάρχει με **Wi-Fi** και **Bluetooth LE (Low Energy)** και υποστηρίζει διάφορες σειρές Espressif SoCs με συνδεσιμότητα Wi-Fi.
- Διαθέτει γρήγορη και φιλική προς τον χρήστη μέθοδο σύζευξης που είναι κατάλληλη για σύνδεση συσκευών «**one-to-many**» και «**many-to-many**», ενώ παράλληλα τις ελέγχει.
- Καταλαμβάνει λιγότερους πόρους CPU και Flash.
- Μπορεί να χρησιμοποιηθεί ως ανεξάρτητο πρωτόκολλο που βοηθά στην παροχή συσκευών, τον εντοπισμό σφαλμάτων και τις αναβαθμίσεις υλικολογισμικού.
- Ο μηχανισμός συγχρονισμού παραθύρων μειώνει σημαντικά την κατανάλωση ενέργειας.
- Χρησιμοποιεί τους αλγορίθμους **ECDH** και **AES-128** που καθιστούν την μετάδοση των δεδομένων πιο ασφαλή.
- Στην κανονική του λειτουργία, έχει εμβέλεια **περίπου 220-250 μέτρων σε ανοιχτό χώρο**, ενώ στην λειτουργία μακριάς εμβέλειας (long range mode) μπορεί να φτάσει **έως και τα 480 μέτρα**.

6.2.2 Μορφή Πλαισίου

Το ESP-NOW χρησιμοποιεί ένα πλαίσιο δράσης ειδικό για τον προμηθευτή για την μετάδοση δεδομένων. Ο προεπιλεγμένος **ρυθμός μετάδοσης (bit rate)** είναι **1Mbps**. Προς το παρόν, το ESP-NOW, υποστηρίζει δύο εκδόσεις, v1.0 και v2.0. Το μέγιστο μήκος ενός πακέτου που υποστηρίζεται από συσκευές v2.0 είναι 1470 byte, ενώ το αντίστοιχο μέγεθος για συσκευές v1.0 είναι 250 byte.

Οι συσκευές v2.0 μπορούν να λαμβάνουν πακέτα τόσο από άλλες συσκευές v2.0 όσο και από συσκευές v1.0. Ωστόσο οι συσκευές v1.0 μπορούν να λάβουν πακέτα v2.0 μόνο εάν το μήκος του πακέτου είναι μικρότερο από 250 byte. Σε περίπτωση που τα πακέτα υπερβαίνουν αυτό το περιορισμό, τότε οι v1.0 συσκευές είτε περικόπτουν τα δεδομένα στα πρώτα 250 byte είτε απορρίπτουν εντελώς το πακέτο. Στην *Εικόνα 9* απεικονίζεται η μορφή του πλαισίου δράσης του προμηθευτή (πρώτο διάγραμμα) και η μορφή του πλαισίου στοιχείων που αφορούν τον προμηθευτή (δεύτερο διάγραμμα).



Εικόνα 9: Μορφή πλαισίου δράσης και πλαισίου στοιχείων του προμηθευτή

- **Category Code:** Ορίζεται στην τιμή (127) που υποδεικνύει την κατηγορία που αφορά τον συγκεκριμένο προμηθευτή.
- **Organization Identifier:** Περιέχει ένα μοναδικό αναγνωριστικό το οποίο είναι τα πρώτα 3-byte της MAC διεύθυνσης του προμηθευτή.
- **Random Values:** Χρησιμοποιείται για την αποτροπή επιθέσεων αναμετάδοσης.
- **Vendor Specific Content:** Περιεχόμενο ειδικά για τον προμηθευτή που περιέχει (τουλάχιστον ένα) πεδία στοιχείων ειδικά για τον προμηθευτή. Για την έκδοση v2.0, αυτό ισοδυναμεί με 1512 ($1470 + 6 \times 7$) byte, ενώ για την έκδοση v1.0 ισοδυναμεί με 257 ($250 + 7$) byte.
- **Length:** Το συνολικό μήκος του Organization Identifier, του Type (τύπος), της Version (έκδοση) και του Body (σώμα). Η μέγιστη τιμή είναι το 255.
- **Body:** Περιέχει τα πραγματικά δεδομένα που θα μεταδοθούν μέσω του ESP-NOW.
- **Version:** Πεδίο ορισμένο στην έκδοση του ESP-NOW.
- **Element ID:** Ορίζεται στην τιμή (221), η οποία υποδεικνύει το στοιχείο που αφορά τον συγκεκριμένο προμηθευτή.
- **Type:** Ορισμένο στην τιμή (4) που υποδεικνύει ESP-NOW.

Όταν το ESP-NOW δεν είναι σε σύνδεση, η κεφαλίδα MAC διαφέρει ελάχιστα από εκείνη των τυπικών πλαισίων. Τα bit του FCS (Frame Control), δηλαδή τα bit FromDS & ToDS, είναι και τα δύο 0. Το πρώτο πεδίο διεύθυνσης ορίζεται στην διεύθυνση προορισμού (destination address), το

δεύτερο πεδίο στη διεύθυνση πηγής (source address) και το τρίτο πεδίο στην διεύθυνση ευρείας εκπομπής (broadcast address), η οποία είναι η **0xFF:0xFF:0xFF:0xFF:0xFF:0xFF**.

6.2.3 Βασικές συναρτήσεις

Προτού γράψουμε οποιαδήποτε εντολή θα πρέπει να κληθούν το **ESP-NOW API** και να **εκκινήσουμε το Wi-Fi**, μιας και το ESP-NOW είναι ένα ασύρματο δίκτυο τύπου Wi-Fi (IEEE 802.11 b/g/n). Αυτό, με χρήση του Arduino IDE γίνεται ως εξής:

```
#include <WiFi.h>
#include <esp_now.h>
```

Τώρα είμαστε έτοιμοι να ξεκινήσουμε.

Συναρτήσεις αρχικοποίησης/απαρχικοποίησης ESP-NOW:

- `esp_now_init()`: Αρχικοποίηση
- `esp_now_deinit()`: Απαρχικοποίηση

Όταν καλείται η συνάρτηση απαρχικοποίησης, τότε όλες οι πληροφορίες των συζευγμένων συσκευών διαγράφονται.

Συνάρτηση επιστροφής στον αποστολέα:

- `esp_now_register_send_cb(dataReceive)`

Με κλήση της παραπάνω συνάρτησης δηλώνουμε την συνάρτηση που έχουμε ορίσει ως συνάρτηση λήψης επιβεβαίωσης, έστω `dataReceive`, η οποία καλείται όταν ολοκληρωθεί η αποστολή ενός μηνύματος από τον αποστολέα προς τον παραλήπτη.

Συνάρτηση επιστροφής στον παραλήπτη:

- `esp_now_register_rcv_cb(dataReceive)`

Με κλήση της παραπάνω συνάρτησης δηλώνουμε την συνάρτηση που έχουμε ορίσει ως συνάρτηση λήψης επιβεβαίωσης, έστω `dataReceive`, η οποία καλείται όταν ληφθεί ένα μήνυμα στον παραλήπτη.

Προσθήκη συζευγμένης συσκευής:

- `esp_now_add_peer(&peerInfo)`: Προσθήκη νέας συσκευής/παραλήπτη (`peer`) με MAC διεύθυνση

Η συνάρτηση αυτή παίρνει ως όρισμα ένα δείκτη προς μία struct (δομή), `esp_now_peer_info_t`, που περιέχει πληροφορίες όπως την MAC διεύθυνση, το κανάλι ή αν χρειάζεται να γίνει κρυπτογράφηση. Το εύρος του καναλιού των συζευγμένων συσκευών είναι από 0 έως 14. Εάν το κανάλι έχει οριστεί ως 0, τα δεδομένα αποστέλλονται στο τρέχον κανάλι, διαφορετικά το κανάλι

πρέπει να οριστεί ως το κανάλι στο οποίο βρίσκεται η τοπική συσκευή. Για την *peerInfo* ισχύουν τα ακόλουθα:

- `peerInfo.peer_addr`: MAC διεύθυνση παραλήπτη.
- `peerInfo.channel`: Ακέραιος αριθμός από 1-14.
- `peerInfo.encrypt`: Τιμή true/false για ενεργοποίηση ή όχι κρυπτογράφησης.

Η συνάρτηση επιστρέφει ένα ακέραιο τύπο που αντιπροσωπεύει την αποτυχία/ επιτυχία της ενέργειας (*esp_err_t*). Εάν η τιμή που επιστραφεί είναι η *ESP_OK*, τότε έγινε επιτυχώς η προσθήκη του peer. Αν επιστραφεί η τιμή *ESP_ERR_ESPNOW_EXIST*, σημαίνει ότι ο peer υπάρχει ήδη. Άλλη πιθανή επιστρέψιμη τιμή είναι η *ESP_ERR_ESPNOW_FULL*, η οποία δηλώνει ότι έχουμε φτάσει στον μέγιστο αριθμό peer που μπορούμε να προσθέσουμε. Η τιμή *ESP_ERR_ESPNOW_NOT_INIT*, στην ουσία μας ενημερώνει ότι έχουμε ξεχάσει προηγουμένως να κάνουμε αρχικοποίηση του ESP-NOW. Τέλος, η τιμή *ESP_NOW_NO_MEM*, δηλώνει ανεπάρκεια μνήμης, πράγμα το οποίο καθιστά την προσθήκη του peer αδύνατη.

Συνάρτηση αποστολής δεδομένων:

- `esp_now_send(mac_addr, data, len)`

Η παραπάνω συνάρτηση στέλνει δεδομένα προς μία συσκευή με την συγκεκριμένη διεύθυνση MAC. Η μεταβλητή `len` αναφέρεται στο μήκος του πακέτου το οποίο εξαρτάται από την έκδοση της συσκευής (v1.0 ή v2.0).

Συνάρτηση ελέγχου ύπαρξης peer:

- `esp_now_is_peer_exist(mac_addr)`

Η συνάρτηση ελέγχου ύπαρξης peer, στην ουσία ελέγχει αν μία διεύθυνση MAC υπάρχει ήδη στη λίστα peer.

Έξτρα συναρτήσεις που αφορούν την βιβλιοθήκη [WiFi.h](#):

- `WiFi.mode(WIFI_STA)`: Θέτει τη συσκευή σε λειτουργία σταθμού (Station Mode).
- `WiFi.macAddress()`: Επιστρέφει την MAC διεύθυνση της συσκευής.
- `WiFi.disconnect()`: Αποσυνδέει τη συσκευή από το τρέχον WiFi δίκτυο (τρέχον AP-access point), χωρίς να απενεργοποιεί τη WiFi λειτουργία (η συσκευή μπορεί να λειτουργεί ακόμα ως STA ή AP). Αν πάρει όρισμα ωστόσο true, τότε αποσυνδέεται και διαγράφει τις αποθηκευμένες πληροφορίες σύνδεσης (π.χ. SSID/Password).

6.2.4 Παράδειγμα επικοινωνίας μεταξύ δύο συσκευών

Παρακάτω, δίνεται αναλυτικά ο τρόπος υλοποίησης της ασύρματης επικοινωνίας μεταξύ δύο συσκευών ESP32, σε γλώσσα C/C++, μέσω του Arduino IDE. Στο συγκεκριμένο παράδειγμα ο αποστολέας (sender) στέλνει στον παραλήπτη (receiver) κάθε δύο δευτερόλεπτα ένα απλό μήνυμα ("Hello World").

Ο κώδικας του αποστολέα:

```
#include <WiFi.h>
```

```

#include <esp_now.h>

//Broadcast MAC Address
uint8_t broadcastAddr[] = {0xFF,0xFF,0xFF,0xFF,0xFF,0xFF};

void setup() {
    // put your setup code here, to run once:
    Serial.begin(115200);
    WiFi.mode(WIFI_STA);
    WiFi.disconnect();

    //First check
    if(esp_now_init() != ESP_OK) {
        Serial.println("Error during initialization.");
        while(true) {delay(1000);}
    }
    //Inclusion of the peer (broadcast)
    esp_now_peer_info_t peerInfo = {};
    memcpy(peerInfo.peer_addr,broadcastAddr,6);
    peerInfo.channel = 0;
    peerInfo.encrypt = false;
    //Final check
    if (esp_now_add_peer(&peerInfo) != ESP_OK) {
        Serial.println("Failed to add peer");
        while(true) {delay(1000);}
    }
    Serial.println("Sender is ready.");
}

void loop() {
    // put your main code here, to run repeatedly:
    const char *message = "Hello World";
    esp_err_t result = esp_now_send(broadcastAddr, (uint8_t *)message,
    strlen(message));

    if(result == ESP_OK) {
        Serial.println("Message sended successfully");
    }
    else {
        Serial.print("Error sending: ");
        Serial.println(result);
    }
    delay(2000); //Wait for 2 seconds for stability
}

```

Ο κώδικας του παραλήπτη:

```

#include <WiFi.h>
#include <esp_now.h>

```

```

//Callout function for receiving a message
void dataReceive(const esp_now_recv_info_t *recvInfo, const uint8_t *
inData, int len) {
    char macStr[18];
    snprintf(macStr, sizeof(macStr),
"%02X:%02X:%02X:%02X:%02X:%02X",
        recvInfo->src_addr[0],recvInfo->src_addr[1],recvInfo->src_addr[2],
        recvInfo->src_addr[3],recvInfo->src_addr[4],recvInfo->src_addr[5]);
    char msg[len+1];
    memcpy(msg, inData, len);
    msg[len] = '\0';
    Serial.print("Received message from ");
    Serial.print(macStr);
    Serial.print(" -> ");
    Serial.println(msg);
}
void setup() {
    // put your setup code here, to run once:
    //Starting serial communication
    Serial.begin(115200);
    //STA mode
    WiFi.mode(WIFI_STA);
    //Disable automatic connection to AP
    WiFi.disconnect();

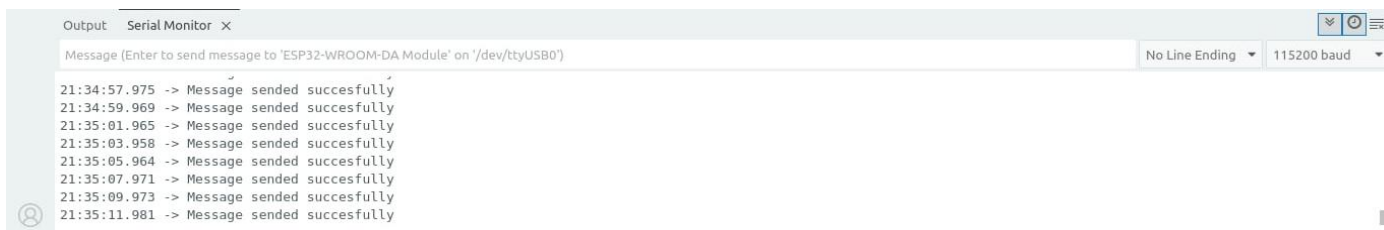
    if (esp_now_init() != ESP_OK) {
        Serial.println("Error initializing ESP-NOW");
        while(true) {
            delay(1000);
        }
    }
    //Registration of the callback
    esp_now_register_recv_cb(dataReceive);
    Serial.println("Receiver ready");
}
void loop() {
    // put your main code here, to run repeatedly:

}

```

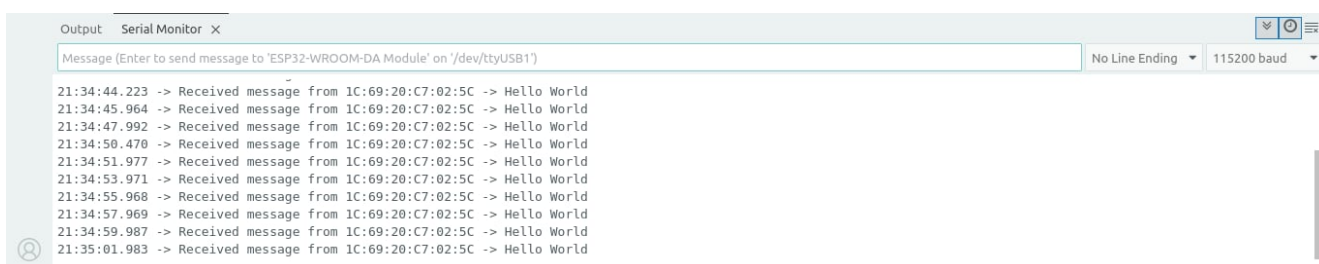
Αφού κάνουμε verify και τους δύο κώδικές μας (πατώντας το εικονίδιο ✓ που βρίσκεται πάνω δεξιά) και αφού κάνουμε upload (πατώντας το εικονίδιο → που βρίσκεται δίπλα από το προηγούμενο εικονίδιο) και τους δύο κώδικες στους δύο μικροελεγκτές ESP32, αν ανοίξουμε το Serial Monitor του Arduino IDE, θα δούμε τα αποτελέσματα που φαίνονται στις εικόνες *Εικόνα 1* και *Εικόνα 2*:

Από την πλευρά του αποστολέα:



Εικόνα 10: Αποτελέσματα στον Sender

Από την πλευρά του δέκτη:



Εικόνα 21: Αποτελέσματα στον Receiver

Με βάση τα αποτελέσματα, βλέπουμε ότι το μήνυμα «Hello World» εμφανίζεται κάθε δύο δευτερόλεπτα στο Serial Monitor του δέκτη, έχοντας μεταδοθεί από τον αποστολέα, του οποίου η MAC διεύθυνση είναι 1C:69:20:C7:02:5C. Κάπως έτσι είμαστε σε θέση να επιβεβαιώσουμε την ύπαρξη ESP-NOW σύνδεσης.

7. Υλοποίηση της VIRSI

Αφού εξηγήθηκαν αναλυτικά στις δύο προηγούμενες ενότητες τόσο η μονάδα αναγνώρισης φωνής όσο και το πρωτόκολλο ασύρματης επικοινωνίας ESP-NOW, σε αυτή την ενότητα θα σχεδιάσουμε και θα υλοποιήσουμε την VIRSI. Θα ενώσουμε δηλαδή την ασύρματη μετάδοση δεδομένων με τις φωνητικές εντολές που έχουμε αποθηκεύσει στο εσωτερικό της VRM. Στο στάδιο της υλοποίησης θα δοθεί έμφαση στην σχεδίαση του ηλεκτρονικού κυκλώματος του ακουστικού κομματιού της VIRSI, του κυκλώματος δηλαδή που θα απαρτίζει τον αποστολέα μας και στα εξαρτήματα/υλικά που εν τέλει χρησιμοποιήσαμε. Για λόγους ευκολίας, το ρομπότ που θα χρησιμοποιήσουμε (δηλαδή ο παραλήπτης μας) θα είναι ένα απλό αυτοκίνητο.

7.1 Υλοποίηση της VIRSI

7.1.1 Υλικά

Για την υλοποίηση της VIRSI χρησιμοποιήσαμε τα ακόλουθα εξαρτήματα:

- **Μονάδα Αναγνώρισης Φωνής (VRM)**
- **ESP32-WROOM32U** (για το ρομπότ)
- **ESP32-S3-WROOM-1N16R8** (για τα ακουστικά)

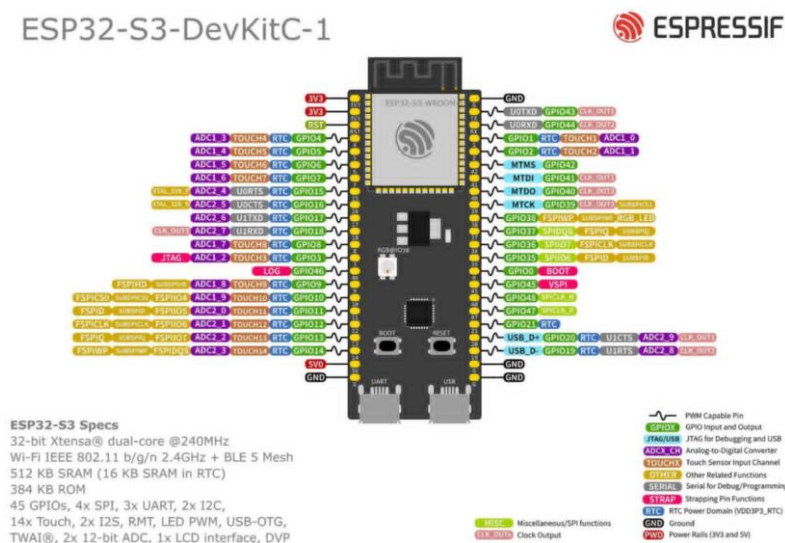
- **Ενισχυτής MAX98357** (δεν χρησιμοποιήθηκε εν τέλει)
- **INMP441 I²S μικρόφωνο** (χρησιμοποιήθηκε ξεχωριστά)
- **Κεραία Folded 2.4G Antenna** (για μεγαλύτερη απόσταση)

7.1.2 ESP-S3-WROOM-1N16R8

Επιλέξαμε την συγκεκριμένη πλακέτα για την σχεδίαση του ακουστικού κομματιού (του αποστολέα) της VIRSI λόγω των ακόλουθων χαρακτηριστικών της:

- **Μνήμη Flash: 16MB (N16)**
- **PSRAM (pseudostatic RAM): 8MB (R8)**
- **Συχνότητα ρολογιού: 240MHz**
- **WiFi: IEEE 802.11 b/g/n (2.4 GHz)**
- **Bluetooth: BLE 5.0**
- **AI Acceleration για ML**
- **45 GPIOs**
- **Περιφερειακές διεπαφές: UART, I²S , I²C, PWM, A/D Converter (12-bit)**

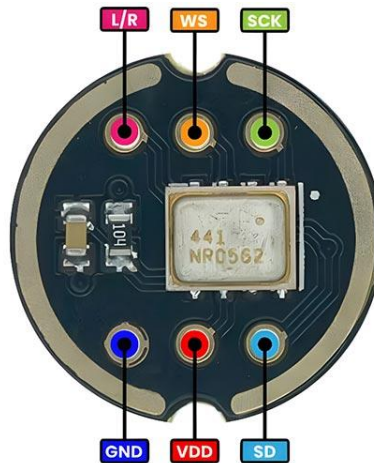
Τα παραπάνω χαρακτηριστικά, καθιστούν την ESP-S3-WROOM-1N16R8 πιο γρήγορη και αποδοτική σε σχέση με την ESP32-WROOM32U και συνάμα ακόμα καλύτερη για χρήση τόσο σε IoT εφαρμογές όσο και σε projects που συνδυάζουν μηχανική μάθηση (machine learning) και computer vision με κάμερες ή audio/voice signals.



Εικόνα 12: ESP32-S3

7.1.3 INMP441 I²S μικρόφωνο

Μπορεί η VRM να διαθέτει το δικό της MEMS μικρόφωνο, με το οποίο μπορούμε να επεξεργαστούμε οποιοδήποτε σήμα φωνής, όμως εμείς θα χρησιμοποιήσουμε το INMP441 μικρόφωνο το οποίο είναι ένα ψηφιακό MEMS μικρόφωνο και αυτό, αλλά χρησιμοποιεί το πρωτόκολλο **I²S (Inter Integrated Circuit Sound Protocol)**.



Εικόνα 13:INMP441 Microphone Pinouts

Τι είναι όμως το I²S;

Το συγκεκριμένο πρωτόκολλο χρησιμοποιείται για ψηφιακό ήχο μεταξύ chip και ελέγχει ένα ή ακόμα και δύο κανάλια μέσω ηχητικού PWM. Χρησιμοποιεί ένα A/D convertor για την μετατροπή του αναλογικού σήματος που δέχεται στην είσοδο σε ψηφιακό σήμα στην έξοδο, του οποίου η ανάλυση καθορίζεται από τον αριθμό των bits (συνήθως μεταξύ 8-32 bits) με τον οποίο πραγματοποιείται η μετατροπή. Το ποσοστό δειγματοληψίας είναι σχεδόν διπλάσιο από την μέγιστη συχνότητα (συνθήκη Nyquist). Επίσης χρησιμοποιεί έναν D/A convertor για να μετατρέψει ξανά το σήμα από ψηφιακό σε αναλογικό. Τόσο στην αρχή όσο και στο τέλος αυτής της διαδικασίας το σήμα ενισχύεται μέσω κάποιου ενισχυτή τόσο στην είσοδο όσο και στην τελική έξοδο, για να φτάσει στον ακροατή. Ένα INMP441 μικρόφωνο, έχει τα εξής pinouts όπως φαίνονται και στην Εικόνα :

- **SCK (Continuous Serial Clock):** Ρυθμίζει την ταχύτητα μετάδοσης των bit. Η τιμή του δίνεται από την σχέση:

Clock Frequency = Sample Rate × Bits per channel × Number of channels (σε Hz), όπου:

→ ***Sample Rate:*** Ρυθμός μετάδοσης δειγμάτων

→ ***Bits per channel:*** αριθμός bits ανά κανάλι

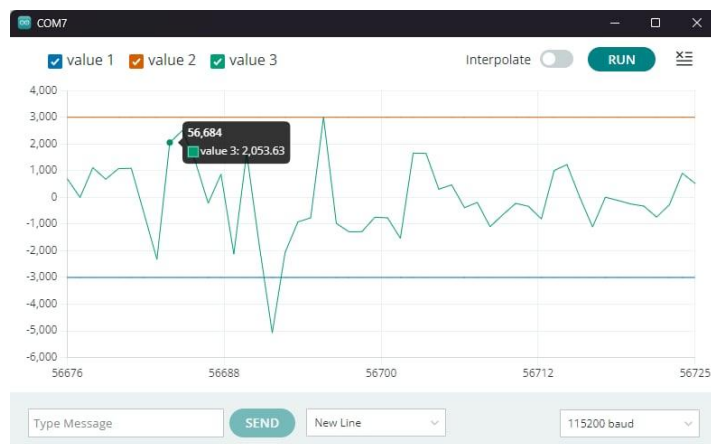
→ ***Number of channels:*** Αριθμός καναλιών

Για παράδειγμα, για ήχο ποιότητας CD, όπου ο ρυθμός μετάδοσης δειγμάτων είναι 44.1 kHz, τα κανάλια είναι 2 και το καθένα από αυτά έχει 16 bits, τότε η συχνότητα του ρολογιού είναι ίση με 1.4112MHz.

- **WS (Word Select):** Καθορίζει ποιο κανάλι ήχου (left ή right) μεταδίδει κάθε στιγμή.
- **SD (Serial Data):** Τα ίδια τα PCM δεδομένα που μεταδίδονται.
- **L/R:** Ανάλογα με το πως το συνδέουμε, έχουμε και το αντίστοιχο κανάλι από το οποίο το μικρόφωνο παράγει έξοδο. Ένα το συνδέσουμε σε κάποια γείωση (GND) τότε λέμε ότι το μικρόφωνο παράγει έξοδο από το αριστερό κανάλι, ενώ εάν το συνδέσουμε να είναι μόνιμα στο HIGH (π.χ. στα 3.3V) τότε η έξοδος του μικροφώνου παράγεται από το δεξί κανάλι.

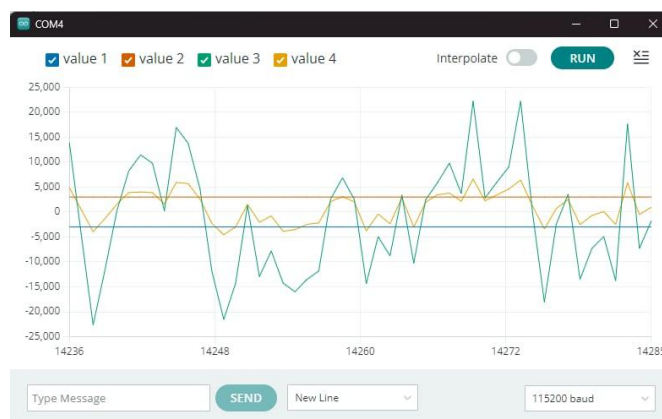
Το INMP441 μικρόφωνο πλεονεκτεί έναντι άλλων, μιας και έχει καλή σταθερότητα, μειώνει παρεμβολές από τα αναλογικά σήματα που δέχεται και έχει μικρό σχετικά θόρυβο. Διαθέτει ένα

αρκετά καλό λόγο σήματος προς θόρυβο (SNR) στα 61dB, η συχνότητα απόκρισής του κυμαίνεται από 60 Hz έως 15kHz.



Εικόνα 14: Σήμα φωνής σε ζωντανή ομιλία.

Στην παραπάνω εικόνα φαίνεται το σήμα ομιλίας μας σε ζωντανή ροή (πράσινη κυματομορφή) όπως το λάμβανε κατά τις δοκιμές μας το INMP441 μικρόφωνο. Οι άλλες δύο κυματομορφές δεν μας δείχνουν κάτι, απλώς υπάρχουν για να μπορέσουμε να «περιορίσουμε» κάπως το σήμα ώστε να μπορέσουμε να το μελετήσουμε πιο εύκολα. Στην επόμενη εικόνα φαίνεται μαζί με το ίδιο σήμα και το συμπιεσμένο σήμα μετά από εφαρμογή της μεθόδου της συμπίεσης (κίτρινη κυματομορφή).



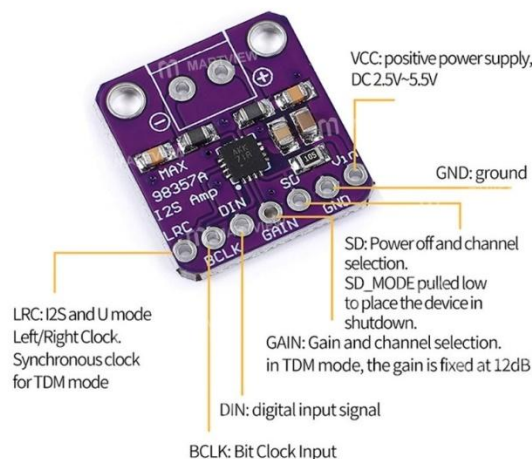
Εικόνα 15: Συμπιεσμένο σήμα σε ζωντανή ροή

Εδώ όπως μπορούμε να δούμε η μέθοδος της συμπίεσης «κόβει» τις πολύ υψηλές εντάσεις που παρατηρούνται κατά την ομιλία μας. Από την άλλη τα πιο χαμηλά σημεία της ομιλίας μας έχουν «περιοριστεί» - κρατηθεί – πιο κοντά στο μέσο επίπεδο. Για αυτό και το σήμα μας δείχνει πιο ομαλό και ισορροπημένο.

7.1.4 Ενισχυτής MAX98357

Ο ενισχυτής είναι ένας **class D audio amplifier** με ενσωματωμένο DAC και λαμβάνει ως είσοδο, **ψηφιακό ήχο I²S** και επιστρέφει στην έξοδό του ένα αναλογικό σήμα το οποίο μπορεί να οδηγηθεί

στην συνέχεια κατευθείαν σε ένα μικρό ηχείο. Ανήκει στην κατηγορία των class D ενισχυτών διότι δουλεύει με **διαμόρφωση PWM (Pulse Width Modulation)**. Τα τρανζίστορ που διαθέτει στην έξοδό του, **διακόπτονται** σε πολύ υψηλή συχνότητα (αλλάζουν δηλαδή κατάσταση από on σε off και το ανάποδο) και «φιλτράρονται» για να δώσουν το αναλογικό σήμα. Τα τρανζίστορ είναι **είτε τελείως ON είτε τελείως OFF**. Έτσι χάνουν ελάχιστη ισχύ για αυτό και μπορούμε να πούμε ότι η απόδοση του ενισχυτή είναι **μεγαλύτερη από 90%**. Επίσης έχει πολύ καλό λόγο σήματος προς θόρυβο (SNR), **περίπου στα 98dB**, που σημαίνει ότι το επίπεδο του χρήσιμου σήματος είναι **πάνω από τον θόρυβο**. Αυτό τους καθιστά κατάλληλους για εφαρμογές IoT, έξυπνου σπιτιού ή ακόμα και «έξυπνα speakers». Στην Εικόνα απεικονίζεται ο MAX98357 ενισχυτής μαζί με τα pinouts του.

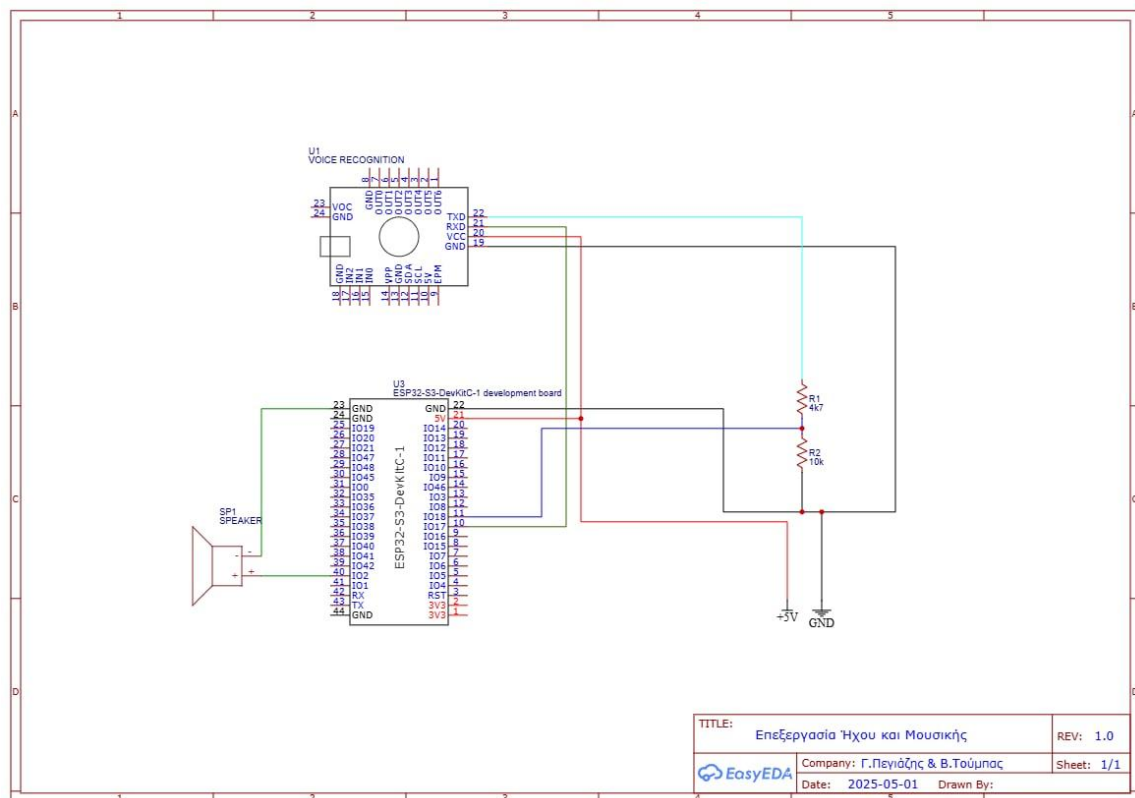


Εικόνα 16: MAX98357 Pinouts

7.1.5 Κυκλώματα Sender & Receiver

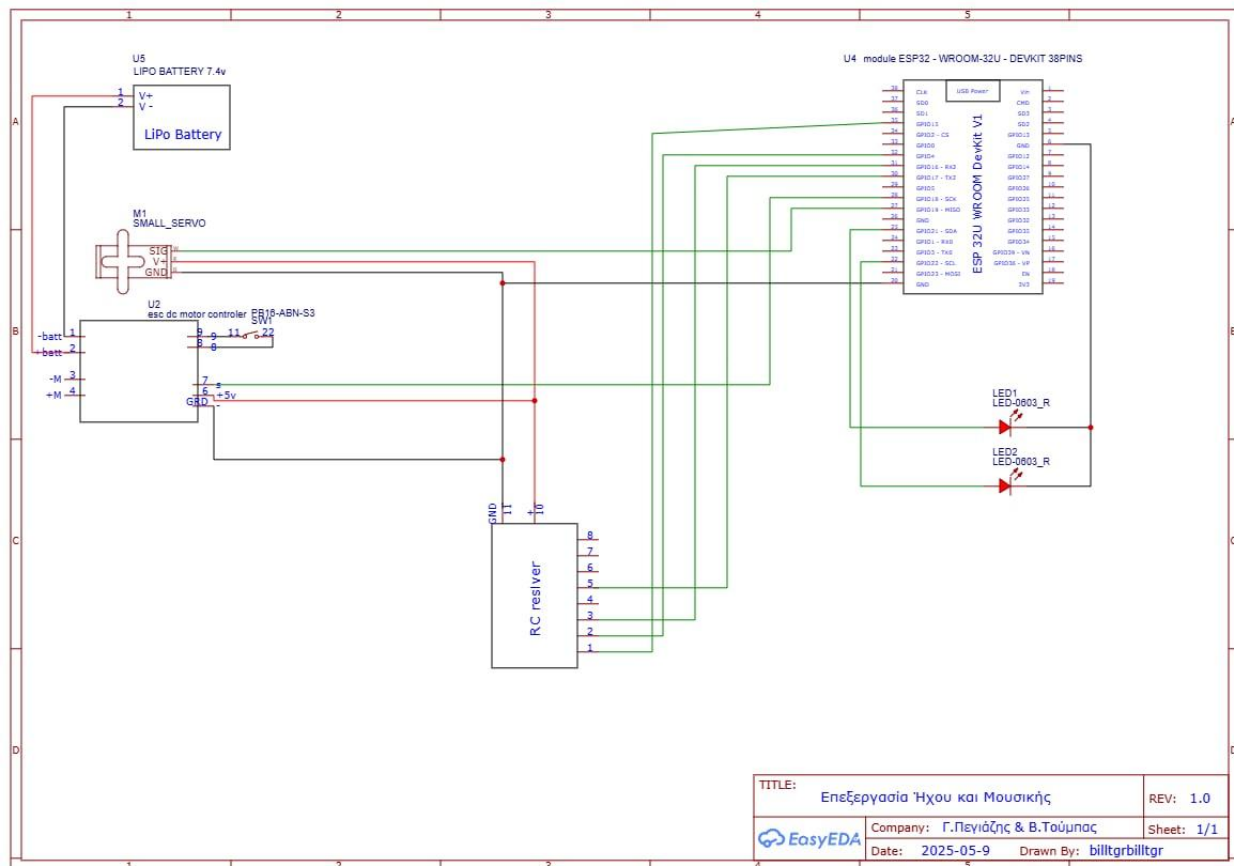
Παρακάτω ακολουθούν σε εικόνες τα τελικά μας κυκλώματα για την υλοποίηση του αποστολέα και του παραλήπτη. Για την απεικόνιση των κυκλωμάτων χρησιμοποιήθηκε το EasyEDA.

Τελικό κύκλωμα του αποστολέα:



Εικόνα 17: Sender

Τελικό κύκλωμα του παραλήπτη:



Εικόνα 18:Receiver

8. Παρατηρήσεις

Κατά την υλοποίηση του project μας ήρθαμε αντιμέτωποι με κάποια -κυρίως τεχνικά- προβλήματα, τα οποία αφορούσαν τη διασύνδεση της VRM με την ESP32. Συγκεκριμένα:

- **Κακή πρώτη βιβλιοθήκη:** Το μεγαλύτερό μας πρόβλημα που αντιμετωπίσαμε είναι ότι η βιβλιοθήκη που χρειαζόμασταν τόσο για την προπόνηση της VRM όσο και για το κομμάτι της αναγνώρισης ήταν πολύ «κακοφτιαγμένη», σε σημείο που κατά την εγκατάστασή της από το GitHub και την κλήση της στον κώδικά μας μέσω του Arduino IDE, **δεν αναγνωριζόταν** (ήταν σαν να μην υπήρχε). Δεν έφταιγε το path στον υπολογιστή μας στο οποίο είχε αποθηκευτεί αλλά το ότι της έλλειπαν κάποια πολύ σημαντικά αρχεία στα οποία αναφέρονταν οι πολύ σημαντικές πληροφορίες για αυτή την βιβλιοθήκη. Φτάσαμε στο σημείο να φτιάχνουμε μόνοι μας αυτά τα αρχεία και να τα αποθηκεύαμε στους κατάλληλους φακέλους της βιβλιοθήκης αλλά και πάλι...
- **Δύο ξεχωριστές βιβλιοθήκες:** Ως συνέχεια του προηγούμενου προβλήματος, μετά από αρκετό καιρό ψαξίματος εν τέλει βρήκαμε ότι υπάρχουν δύο ακόμη βιβλιοθήκες με τις οποίες μπορούσαμε να δουλέψουμε. Η μία βιβλιοθήκη χρειαζόταν για την **προπόνηση** της πλακέτας και η άλλη για την **διασύνδεση της πλακέτας με την ESP32**. Και αυτό, γιατί κανονικά η VRM V3 είναι φτιαγμένη για Arduino και όχι για ESP32 μικροελεγκτές.
- **Διαιρέτης τάσης για λειτουργία στα 5V:** Επειδή στην αρχή τροφοδοτούσαμε την VRM V3 στα 5V, χρειάστηκε να φτιάξουμε ένα διαιρέτη τάσης με τον οποίο κατεβάζαμε την τάση στα

3.3V για τα pins της ESP32-S3, μιας και πρώτον μας το πρότεινε η παλιά μας βιβλιογραφία και δεύτερον τα pin των ESP32 λειτουργούν **μόνο στα 3.3V**. Για αυτό και μπορείτε να τον δείτε στο κύκλωμα του αποστολέα στην Εικόνα 16 πιο πάνω. Εν τέλει όμως μετά από δοκιμές είδαμε ότι το project λειτουργούσε πολύ καλά στα 3.3V και χωρίς αυτόν. Για τον διαιρέτη τάσης χρησιμοποιήσαμε δύο αντιστάσεις των **10kΩ** και **4.7kΩ** αντίστοιχα μεταξύ του πομπού της VRM V3 και του δέκτη της ESP32-S3.

Για λόγους κυρίως ασφάλειας το σύστημά μας έχει και λειτουργία RC, δηλαδή με χρήση τηλεχειριστηρίου. Ακόμη, παρατηρήσαμε ότι το σύστημά μας εν τέλει είχε πολύ καλή συμπεριφορά σε πολλές δοκιμές μας. Με αφορμή αυτή την συμπεριφορά του συστήματος και λόγω χρόνου πήραμε την απόφαση το κομμάτι της συμπίεσης να το δούμε λίγο ξεχωριστά και να μην υπερφορτώσουμε τον αποστολέα μας (ακουστικό) με δύο μικρόφωνα. Το gooseneck μας κάλυψε πάρα πολύ και μαζί με την κεραία που βάλαμε πλέον μπορούμε να χειριστούμε το ρομπότ μας και από πιο μακρινή απόσταση με το ESP-NOW. Τέλος, τον ενισχυτή MAX98357 δεν τον χρησιμοποιήσαμε εν τέλει σε κάποιο κομμάτι απλώς μπορέσαμε να τον δούμε λίγο και αυτόν στην πράξη.

Όλους μας του κώδικες μπορείτε να τους βρείτε στο repository μας στο GitHub στον ακόλουθο σύνδεσμο: <https://github.com/GeorgePeg/VIRSI>

9. Πηγές

Παρακάτω ακολουθούν όλες μας οι πηγές που χρησιμοποιήσαμε για το project της VIRSI:

Οι βιβλιοθήκες που εν τέλει αξιοποιήσαμε:

https://github.com/Muehli-Industries/ElechouseVR3_ESP32/tree/main

https://electronoobs.com/eng_arduino_tut165.php

Η βιβλιοθήκη που είχαμε στην αρχή:

<https://github.com/elechouse/VoiceRecognitionV3>

<https://blog.frankvh.com/2022/02/21/esp32-support-for-elechouse-voice-recognition-module-v3/>

Για το ESP-NOW:

<https://www.espressif.com/en/solutions/low-power-solutions/esp-now>

<https://www.cloudflare.com/learning/ddos/glossary/open-systems-interconnection-model-osi/>

https://docs.espressif.com/projects/esp-idf/en/latest/esp32c3/api-reference/network/esp_now.html

Για το robot:

https://www.youtube.com/watch?si=fhnLfbmc_sz48225&v=HpChhPt1CCQ&feature=youtu.be

Για την VRM V3:

https://www.elechouse.com/elechouse/images/product/VR3/VR3_manual.pdf

Για τον I²S ήχο και το INMP441 μικρόφωνο:

<https://en.wikipedia.org/wiki/I%C2%B2S>

<https://hackaday.io/project/162059-street-sense/log/160705-new-i2s-microphone>

<https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/peripherals/i2s.html>

<https://github.com/ikostoski/esp32-i2s-slm>

<https://invensense.tdk.com/wp-content/uploads/2015/02/INMP441.pdf>

Για τον ενισχυτή MAX98357:

<https://www.analog.com/media/en/technical-documentation/data-sheets/max98357a-max98357b.pdf>

Για την μέθοδο της συμπίεσης χρησιμοποιήσαμε την διάλεξη «Ψηφιακή Επεξεργασία Ηχογραφήσεων» από το e-class του μαθήματος «Επεξεργασίας Ήχου και Μουσικής». Τέλος, για πληροφορίες για το σήμα της φωνής χρησιμοποιήσαμε το 4^ο κεφάλαιο από το σύγγραμμα του μαθήματος «Ψηφιακή Επεξεργασία Φωνής – Θεωρία και Εφαρμογές».

ΤΕΛΟΣ